

NOTES

HTML → Structure of Web App
 JS → Function of Web App (REACT)
 CSS → styling

Date

Main Work of React:- Takes your data and shows it on the screen

Frontend:- React.js

Backend:- Node.js / Express

Full Stack:- Next.js

Things to Study:-

- System Design
- React Concepts
- Rendering Models

Server-First Architecture:- All the heavy loading JS codes are run in server and a HTML file is to Client browser. Client Machine Independent, faster rendering

Role of Developer in AI era:-

- Define architecture
- Review AI-generated output
- Focus on design decisions
- Validate correctness & performance

User searches a web App $\xrightarrow{\text{HTTP GET}}$ Server sends the HTML file $\rightarrow$ JS file created in Client browser
 { React is a JS framework to handle functional requirements of web Apps }

NOTES

Concepts
 REACT COMPONENTS

Date 10/08/26

Chapter 1:- The Component Model

Building blocks of React App

JS functions: takes props and returns UI markup (JSX)
 What to draw on your screen

→ Elements are static.

→ doesn't remember

Q) If React elements are static, when a user visits a website and changes its appearance to dark mode, the performance must be gone when he visits the site again?

A) User clicks on dark mode $\rightarrow$ state changes in respective component $\rightarrow$ the choice is saved in the browser cache

* The Long Term Memory

next time he visits, the state of React

Component gets updated by cache.

→ The Flow of Data is Unidirectional:- Top to Bottom

Information

Root Component (parent)

Smaller component (children)

Q) Why Unidirectional?

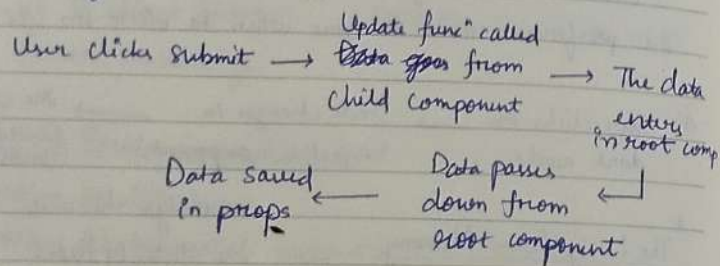
A) If there is any change in data, we must look to the parent. This makes finding bugs easier. We can't check each and every component in checking errors

NOTES

Date

Q) There is a page to submit your info. You filled all the details & clicked 'Submit' button. How is the info saved to respective props? Data moves from Top to Bottom i.e. main/root component to children components. 'Submit' is a child component, it cannot pass info to the main component!!

A) The main component creates a 'update' function along with data.



• All this happens when the page refreshes when we click a submit button

useState:- Updates the props and renders the component once again

NOTES

Date

Chapter 2:- Local State & Immutability (useState)

→ There are two ways of writing React code:-

- Long Class Components: Similar to C++ Objects with all the functionalities like mutability of props
- Functional Class Components: Short and simple lightweight JS functions. Doesn't have 'memory' functionalities

Hooks:- Plugins to functional components to have their own memory like Class components.

useState Hook:- Gives temporary memory to components

const [count, setCount] = useState(0)

memory function after update it renders the return func of comp again

• setCount(count + 1)

This erases the current count value and updates it with the new value

Q) Why can't I just do count++ and display it on the screen? A) The variable may get updated but React won't run the return function until we reload it.

NOTES

Date

Chapter 3: → Life Cycle and Resource Management (useEffect)

• useEffect is a hook that enables React to do more than just displaying functional JS components on screen. It's called Side-Effects

• 3 components of useEffect:

- (a) Setup function: Code we want to run
- (b) Cleanup function: It undoes the setup
- (c) Dependency array: Tells React when to re-run the setup

```
useEffect() => {
```

```
  // Setup function
```

```
  return () => {
```

```
    // Cleanup function
```

```
  };
```

```
}, [// Dependency array];
```

NOTES

Date ...11/08/26...

Chapter 4: → Event Delegation (Callbacks)

→ The parent component passes it function as a prop to its child component, such that whenever a new data comes to the child component it can call back the parent component & the data will be moved from Top to Bottom unidirectionally

Chapter 5: → Virtual DOM → Document Object Model

→ REACT doesn't make changes to our original Model bcz it'll be slow. It creates a new virtual DOM tree & makes the new changes from Virtual DOM to Original DOM

Things to Ask Manager

- High: Claude, meet the team → manager told to look thru
- ✓ Mid: HWA overview PPT, DWC L3 work (Planned by Nanda)
- Low:

Workload Automation Overview

Playlist:- HWA v10.1.0 → Bridge IT and Business

• What's contextual help?

→ Integrates file transfer & data transfer in one workflow. Moving files between agents

→ AI powered anomaly detection for workloads

IBM Workload Scheduler

Same as Workload Automation

AIDA (AI Data Advisor) :->

Uses ML to forecast time series KPIs

1. Automatic Planning and Scheduling
2. Scheduler builds the plans from description of workload
3. AIDA:- Fast and simplified data-driven decision making

Workload Automation v10.2.7

→ 'started' alias: A new filter to view started jobs only

14/08/26

Workload Automation v10.2.8

how is v10.2.8

→ Run Cycle based job stream
different than
Submission in Workload Automation 10.2.7?

→ Bulk Actions

- Bulk Draft & Undraft
- Bulk NOP & UNNOP : NO operation

VERA:- Validation Environment for Readiness
Assessment
→ Eliminate Manual Testing

NOTES

JAVA Basics

Date 19/08/26

* Abstraction in JAVA: →

Interface Drivable {
CAN-DO void startEngine();
}

→ 'IS-A'

abstract class Vehicle {

abstract void refuel();

void honk() { ----- };

}

class Tesla extends Vehicle implements Drivable {

public void startEngine() { ----- };

void refuel() { ----- };

}

• All methods in a interface are abstract by default

• All abstract methods need to be included in child classes

• Multiple extend not allowed but multiple implement allowed

→ implemented
→ extended

* Method Overloading :- Same named functions in a class with different parameters

* Method Overriding :- Same named functions in parent & child classes. overriden

* 'Super()' is used to call the function of parent class

JS

~~JS~~ Fundamentals

Date 24/08/26

strict mode → 'use strict'; { beginning of code }
→ minimize silly errors by developer such as typo

Functions help to write DRY code
Don't Repeat Yourself

* use Function Declaration always
↳ like C++

Arrays → defined in square brackets

* ARRAYS CAN BE MUTATED EVEN IF DECLARED IN 'CONST' KEYWORD

* ARRAYS CAN HOLD VALUES OF DIFF. TYPES

→ .push(), .unshift(), .pop(), .shift(),
.indexOf(), .includes()

→ Parameters of an object can be accessed using dot (.) and square brackets [] like Pandas

Allows concatenation of expressions

NOTES

Date

• for-loop has 3 parameters → (initialization, condition, update)

• while-loop has 1 parameter → condition

↳ initialization before loop

↳ update inside loop

• Destructuring:- converting objects into individual types

`const [a, b, c, ...z] = array`

• Spread:- opening up an array without creating new variables

• Rest:- closing/collecting elements in an array

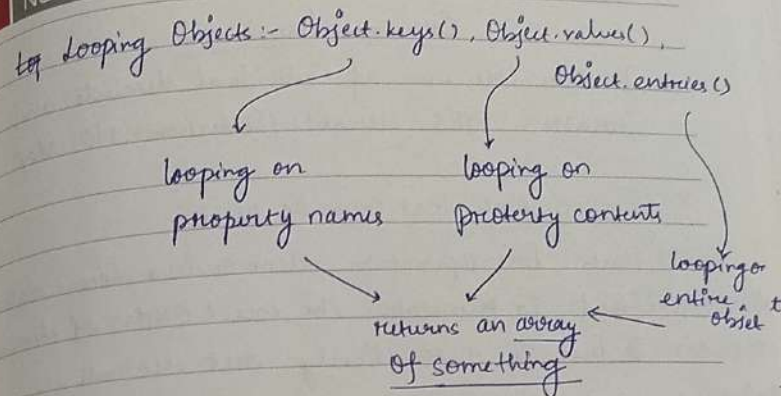
• OR → gives first truthy value

AND → gives first falsy value second operand only when the first one is truthy

(?.) Optional chaining:- checks if a particular value we are trying to access is valid/defined

NOTES

Date 25/02/26



ADVANCED JS

1. Closures:- The nested inner functions can access the variables of outer function

eg:- `function createCounter() {`

`let count = 0`

`return function () {`

`count++;`

`console.log(count);`

`}`

`const counter`

`= createCounter`

`// 1 → counter();`

`// 2 → counter();`

`// 3 → counter();`

* counter is basically the inner func of createCounter

Collections Framework

1. Is our collection of individual elements and iterable? YES: Iterable/Collection NO: Map

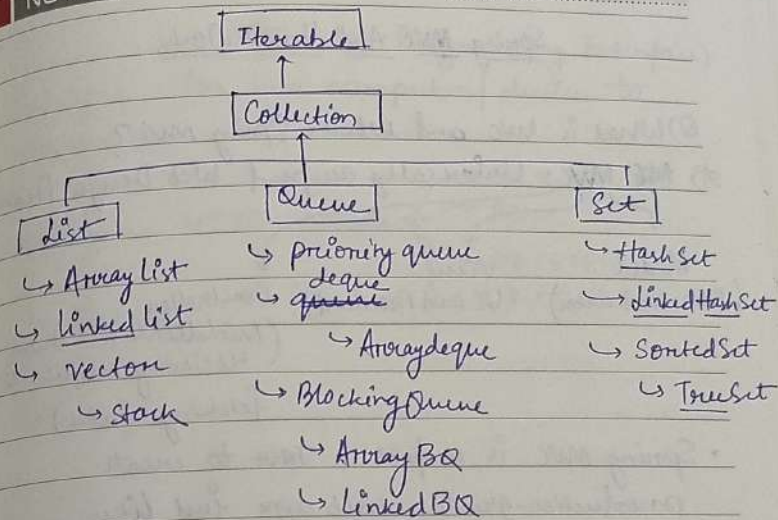
2. Three Prefixes that can be used:-

- (a) Hash: fast operation, items order doesn't matter
- (b) Linked: to remember the exact order of elements
- (c) Tree: to automatically sort elements

Q) Difference b/w hash & map?

As map is the idea of storing elements in key-value pair

hash is the mechanism/mathematical expression that takes 'key' as an input and calculates the address where value is stored, making retrieval $O(1)$

Map

- ↳ HashMap
- ↳ LinkedHashMap
- ↳ Sorted map
 - ↳ TreeMap

* Spring Model: Data, Business Logic

* Spring Controller: Dispatcher Servlet, Handler Mapping, @RestController

* Spring View: View Resolver, View

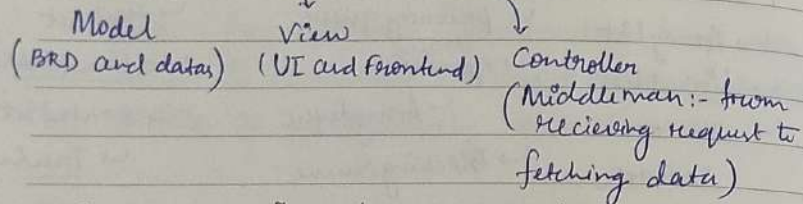
NOTES

Date

Spring MVC And How it Works

Q) What is MVC and what's Spring MVC?

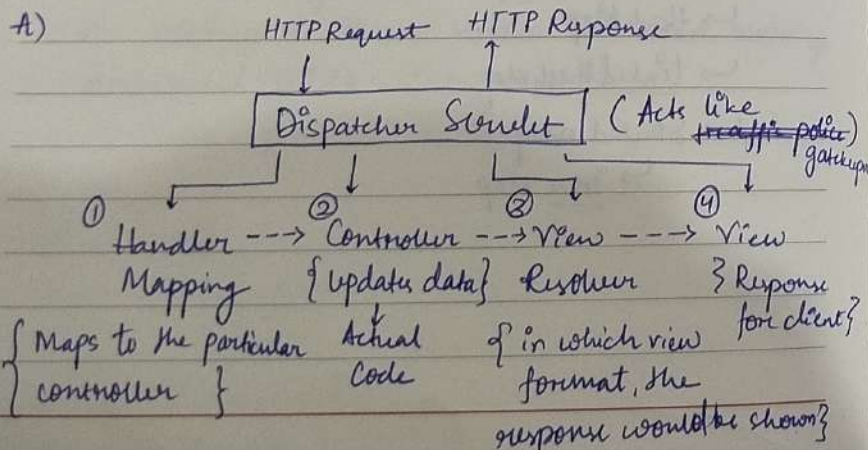
A) ~~THE~~ MVC: Universally accepted Web Design Framework



- Spring MVC is a tool in ~~JAVA~~ to create production-grade web apps just like React for JS

Spring

Q) How does MVC work?

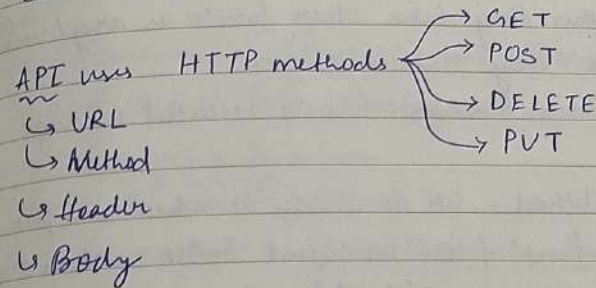


NOTES

Date

API (App Programming Interface)

Interface for two computers / devices to communicate over internet



NOTES JS Advanced cont...

11FE: Immediately Involved
Function Expressions

Date 27/08/26

1. Closure:- Nested inner func's can access variables of outer func's
 2. HOF:- Higher Order Functions
 - (a) Either they take other func's as arg/^{input}
 - (b) Or they return a func'
- First Order functions:- Func's treated as args

3. Call Method:- We don't use '()' when we pick up a funcⁿ from an object inside another object so that we can 'call' it when we want, in our second object. By this way, the funcⁿ gets stored and not gets ~~is~~ implemented.

e.g. - ~~the~~ const book = lufthansa. book;

Stored funcⁿ

↳ this is
a funcⁿ from

book call (enrollings, ..., ...) 'lufthansa obj.

bez we used 'this' keyword in 'book'
and the ^{**}first arg inside call points
to that object

↳ Bind Method

- ↳ first arg: 'this' object name
- ↳ returns new function

Spring MVC Project

Date 01/09/26

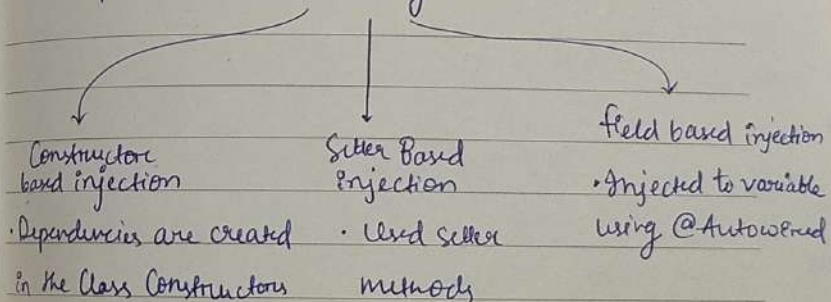
Some Spring Core fundamentals: →

1. Invasion of Control:- Spring manages the creation and management of objects instead of our app code.

↳ IOC Container :- {Application Context} Mechanism that controls ToC. Working: →

- Read config. instruction & message
- Create necessary obj.
- connect them based on their requirements

2. Dependency Injection:- Spring injects the required dependencies directly into the object, rather than letting it build on its own. This reduces the dependency on components inner working.



NOTES

Date

3. Spring Beans:- Spring objects managed by IOC container
 → if we use 'new' keyword to build an object it is not a Spring Bean.
 → Register class as a Bean using @Component

x

Maven:- Project Architect, Repo Management, Automation

Stage 1: BRD

Stage 2: Architectural Design & Tech Stack

Stage 3: Design Pattern & Flow

Stage 4: Code

HLD: Architectural Patterns

LLD: Design Patterns

Date 02/09/26

NOTES

Architecture Patterns (HLD)

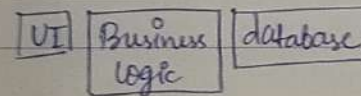
High level structure of our project Subsystems and module interaction

- ↳ MVC, Microservice, Layered Architecture, Event Driven Architecture, ~~SOA~~, SOA Client Server Architecture

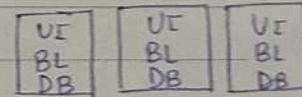
1. Layered Architecture (n-tier): Standard for organising horizontal functionalities

- ↳ One Bdg Data Storage that handles saving data for User Profiles, Products & Orders

vertical functionalities:- User Profile services has its own dedicated UI, business logic & database



Common for all the services



Separate for separate services

NOTES

Date

2. Client-Server Architecture: Client gives request, Server provides response
3. Event Driven: Real Time System & graphical user interface. Responds to asynchronous events - triggered by user actions or state changes
4. Microservices: Organising vertical functionalities. It decomposes monoliths into isolated units creating isolated ^{components} and keeping it operate on their own which can be ~~used~~ integrated. Used for cloud environments and independent scalability

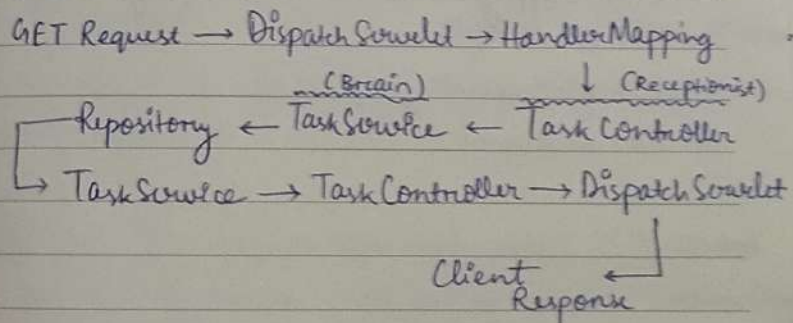
CSR: Presentation layer, Business logic layer, Data Access layer

NOTES

Dev Tracker phase 01 Date

1. BRD:- A Spring MVC application that exposes REST APIs to CRUD (create, read, update, delete) tasks. Data stored in JAVA collections
2. Architecture: MVC, Tech Stack: Spring MVC & APIs
3. Design Pattern: Client Service Repository (CSR)
 - a) TaskController → MVC App Controller. Intercepts the request & maps it to a Java Method
 - b) TaskService → MVC Model. Contains business logic
 - c) Repo ~~base~~ → MVC Model. Data
 - d) Response → MVC view. Returns the data as JSON back to the client

Flow:-



NOTES

@Component: Make a class JAVA Bean
 @Service: Specialized Business logic keyword of
 @Component Date

ANNOTATIONS

- i) @Component :- Spring managed JAVA class (Beans)
- ii) @Service :- Specialized @Component + Business logic
- iii) @Repository :- Mark a class as DAO (Data Access Object)
- iv) @Controller :- MVC web controller to manage HTTP
- v) @Autowired :- Marks a field/method to inject Bean dependency
- vi) @Configuration :- Declares that a class contains config methods
- vii) @Bean :- To manually instantiate & return Spring Beans

viii) Spring Boot Application

- i) @RestController :- @Controller + Returns in JSON
- ii) @RequestMapping :- Defines URI path {Handler Mapping}
- iii) @GetMapping :- Read, @PostMapping :- Write

NOTES

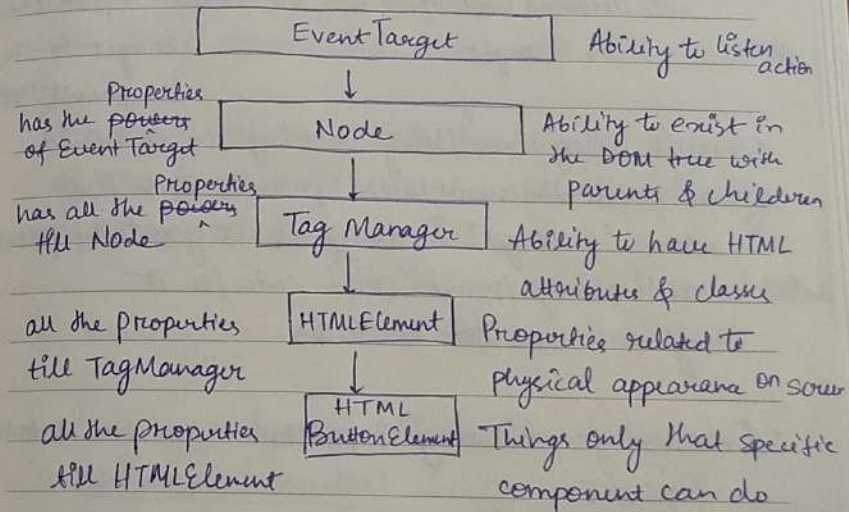
DOM & Advanced JS

Date 03/09/26

DOM Concepts (Document Object Model)

→ Web API that gives object oriented interface to your HTML document

DOM Hierarchy :- Helps us to categorise a component on the screen. ~~From~~ The more you go down, the more specific you go to that component



"DOM is the live object model that enables JS to control the physical appearance (css) and component structure (html) component structure (HTML) & functionalities"

event: 'e'

NOTES

Date

- Event: Any trigger on a DOM element
- EventHandler: DOM Guard. Whenever a trigger occurs, eventhandler runs the code

• Event Propagation: Capturing, Target, Bubbling!

travelling from document root down to the target reaching the target travelling from the target to the document root

- Event Delegation: Using bubbling we can add an event listener to the parent such that when 'e.target' is triggered, it goes to the parent and changes/runs some code for it. They are used for callbacks.

- Debouncing: - Running a function after ~~an~~ certain TIMESET

- Throttling: - Executing a function at fixed time intervals

NOTES

JS final

Date ...09/09/26

JS is Asynchronous in nature

When a line of code is running in the background, it executes next lines of code

Issuing being async: - If we are downloading a file in one line and ~~to~~ updating it in the next, our 2nd line may get ^{run} executed before the doc being downloaded, which will throw an ERROR

Solution: - Add a callback funcⁿ in the args of main function to ensure execution & loading is complete

→ This looks messy in nested callbacks
Solution: - Promises

Promises (Singer-Fans Analogy)

new Promise (executor)

State: "pending"
Result: Undefined

→ resolve(value)

"fulfilled" value

→ reject(error)

"rejected" error

Augs of emulsion:- resolve, reject
 ↓ ↘
 result error

NOTES

```
eg:- let promise = new Promise(function(resolve, reject){
    setTimeout(() => resolve("done", 1000);
    });
```

After this, the function will run & give ~~the~~ the value generated to forward

→ Executor should call only one resolve on one object. Any state change is final

→ Reject a task using `Exception` objects.

→ We can call resolve/reject function on our code manually inside it

→ 'state' and 'results' of Promise objects are internal

* promise.then(^{abort}~~reject~~); Successful completion

* premise catch (alert): Catching errors

NOTES

REACT FUNDAMENTALS

Date 10/09/26

Component-based : Modern Web Development

VI:- HTML + JS + CSS
(markup) (functional) (style)

JSX allows to write HTML like code in JS functions

Benefit: Binds layout and logic of a component together

→ HTML is a static language. But JSX is designed for dynamic apps where UI reacts to changing data and logic

- JSX codes cannot be compiled in browsers. It's converted to ^{machine readable} JS first and then implemented.

L) Buildw:- Packages app for browser (Vite)

It ~~converts~~^{Optimizes} our JSX and CSS files to ~~browser~~^{readable readable code} into compact files

Build tool: Compiler { Converts JSX to JS }

NOTES

Date 16/09/26

- Adding 'import' method for using an image for a UI is much more accurate than writing `src="/assets/img.png"`. Because `import` is a special keyword that tells our UI is dependent on this image to load. The image components gets added to the list of Dependency Graph of our page.
- Props: Properties of a ~~from~~ component. Destructure the Props Object.
- Event-Handler using "on" prefix before function name passed as props. eg:- `onClick`, `onSubmit`.

useState → Render → useEffects

NOTES

Date 16/09/26

~~useEffects~~useState (hook) → maintains state of prop`const [value, setValue] = useState(initialValue)`

working:- changes the variable value and re-renders that particular component

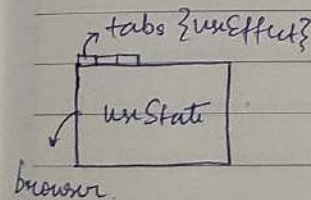
useEffect (hook) → maintains Synchronization

Synchronization:- Matching Internal React State with external system

If an external system (a server, the browser, a timer) need to be updated to reflect this new reality? use "useEffect"

prop updates → useState requests → Browser does that
to render the component

based on newState



run useEffect
to clean up and sync
the outside world
with the new data